

COMP1549: Advanced Programming

Part a: Introduction to COMP1549

Part b: Fundamentals of Programming in Java

Dr Taimoor Khan

WWW

- [Personal](#)
- [Academic Centre of Excellence in Cyber Security Research \(ACE-CSR\)](#)
- [Academic Centre of Excellence in Cyber Security Education \(ACE-CSE\)](#)

15th January 2026 - Week 1

Introduction to COMP1549



Module structure



- **Lecturers:** Taimoor & Markus
- **Tutor:** Najma & Muqaddas
- **Lectures:** 10 key topics in 10 weeks x 1 hrs per week (+1 coursework demonstration week)
 - **When*:** Thursday 9am-10am, 10am-11am, 1pm-2pm
 - **Where:** KW315 (LT), QA280 (LT), 11_0003 (LT)
- **Labs:** 10 topics in 10 weeks x 2 hrs per week
 - **Aim:** To practice and implement the learned concepts during lectures, and from the supporting material
 - **When* and where:** KW103 A and KW103 B
 - **Supported tool:** Eclipse
 - **Solutions:** Only provided in collaboration with students on demand during lab



* For individual slot, please check your timetable

To enable you to learn

- Art of programming based on
 - Design thinking that helps to develop
 - ❖ Reliable
 - ❖ Secure
 - ❖ Efficient and
 - ❖ Maintainable code

Design thinking core principle include

- Challenge your assumptions
 - Why do we need it?
 - ❖ There must be a reason not to avoid it
 - How do we use it?
 - ❖ There must be core benefit of doing it
 - ❖ You must be aware of the side effect of the use

Only way to achieve that is to understand concepts and practice them

- Wk1 : Fundamentals of Programming in Java
- Wk2: Revision of inheritance, polymorphism and abstraction in Java
- Wk3: Threads (Concurrent and Network programming)
- Wk4: Generics
- Wk5: Secure Programs
- Wk6: Java Modeling Language – Open JML
- Wk8: JUnit and Version Control (Markus)
- Wk9: Design Patterns (Markus)
- Wk10: Lambda (ORM) (Markus)
- Wk11: Reflection (Markus)
- Wk12: Demonstrations

● Assessment



➤ 1 Coursework (100%)

- ❖ *A group (1-6 students) project to develop an application in Java using various concepts taught in this module*
- ❖ *Will be released on **Jan. 19th**, and is due on **Mar. 30th***
- ❖ *Project demonstrations will take place on **April 2nd***
- ❖ *All students must complete their group details online by **January 30, 2026***
 - *with IDs of their group members, if they want to work in a group*
 - *or your own ID, if you want to work individually*
- ❖ *We will form groups for the remaining ones and finalise others on **January 30, 2026***



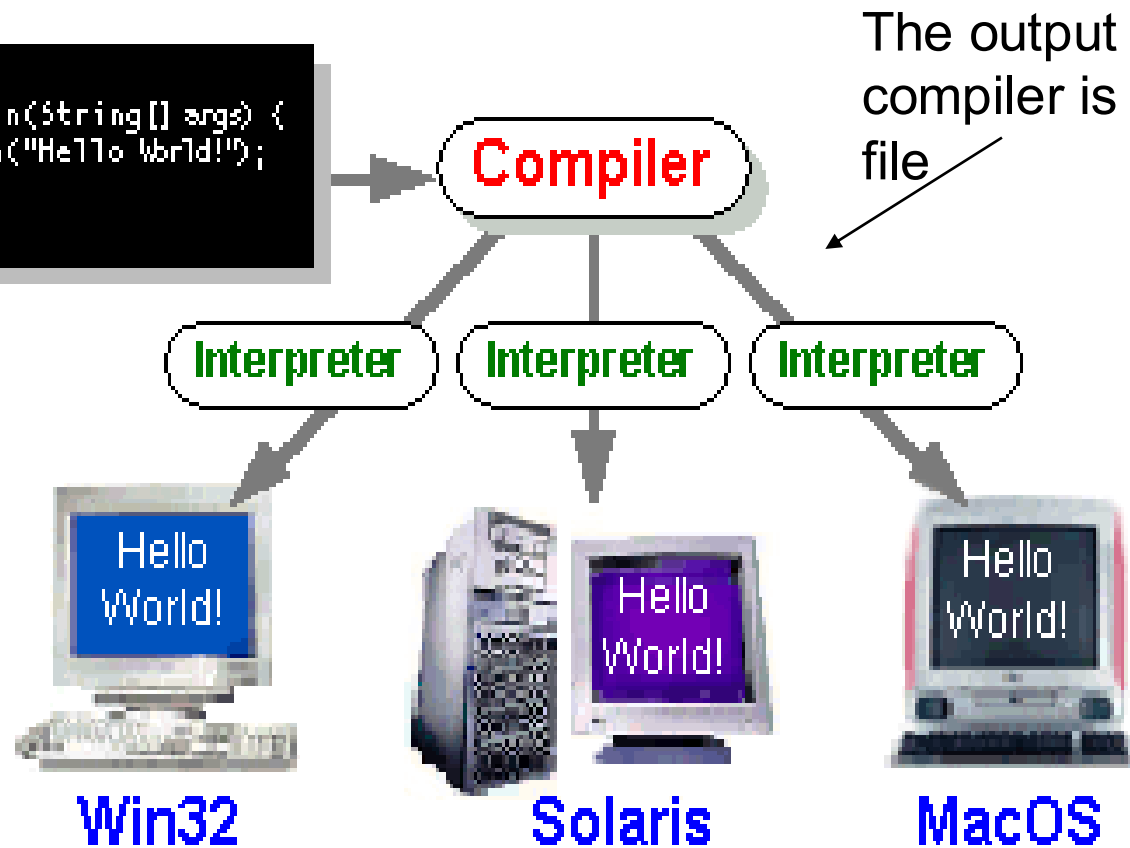
Introduction to Java



Java Program

```
class HelloWorldApp {  
    public static void main(String[] args) {  
        System.out.println("Hello World!");  
    }  
}
```

HelloWorldApp.java



The output of the compiler is .class file

The Interpreter's are sometimes referred to as the Java Virtual Machines

- Java is a "pure" Object Oriented Language
 - encapsulation, inheritance, and polymorphism
 - all code must be contained in a class
 - no free functions (functions that do not belong to some class) like C++, although someone who wants to write messy Java code certainly can
 - Is OO the best programming paradigm?

First Program - HelloWorld.java

```
/**  
 * A simple program  
 */  
  
public class HelloWorld  
{  
    public static void main(String[] args)  
    {  
        System.out.println("HELLO COMP1549!");  
    }  
}
```

- All code part of some *class*

```
public class Foo
{
    //start of class Foo
    /*all code in here!*/
} // end of class Foo
```

- The code for class Foo will be in a file named Foo.java
 - just a text file with the .java extension
 - a class is a programmer defined data type
- A complete program will normally consist of many different classes and thus many different files
- Different classes related to a specific task can be bundled in the same folder called package
 - The package serves as a library and can be reused in other programs

- Syntax errors / Compile errors
 - caught at compile time.
 - compiler does not understand or allow
- Runtime errors
 - something “Bad” happens at runtime
 - Java breaks these into Errors and Exceptions
- Logical errors
 - program compiles and runs, but does not do what you intended or want

Basic Features of Java



- Data Types
 - primitives
 - classes / objects
- Expressions and operators
- Control Structures
- Arrays
- Methods
- Programming for correctness
 - Assertions
 - Pre and post conditions – **Week 6**

Java Data Types

- Primitive Data Types

- byte short int long float double boolean char

```
//dataType identifier;  
int x;  
int y = 10;  
int z, zz;  
double a = 12.0;  
boolean done = false, prime = true;  
char mi = 'D';
```

- stick with int for integers, double for real numbers

- Classes and Objects

- pre-defined or user defined data types consisting of constructors, methods, and fields (constants and fields (variables) which may be primitives or objects)

- Class is synonymous with data type
- Object is like a variable
 - The data type of the Object is some Class
 - referred to as an *instance of a Class*
- Classes contain
 - the implementation details of the data type
 - and the interface for programmers who just want to use the data type
- Objects are complex variables
 - usually, multiple pieces of internal data
 - various behaviors carried out via *methods*

- Declaration - DataType identifier
`Rectangle r1;`
- Creation - new operator and specified constructor
`r1 = new Rectangle();`
`Rectangle r2 = new Rectangle();`
- Behavior - via the dot operator
`r2.setSize(10, 20);`
`String s2 = r2.toString();`
- Refer to documentation for available behaviors (methods)

- ◉ Java has a large built-in library of classes with lots of useful methods
- ◉ Ones you should become familiar with quickly
 - String
 - Math
 - Integer, Character, Double
 - System
 - Arrays
 - Scanner
 - File
 - Object
 - Random
 - [Look at the Java API page](#)

- Using an import statement by an import keyword
- Packages and classes can be imported to another class
- Does not actually import the code (unlike the C++ `include` preprocessor command)
- Statement outside the class block

```
import java.util.ArrayList;  
import java.awt.Rectangle;  
public class Foo{  
    // code for class Foo  
}
```

Expressions

- Expressions are evaluated based on the precedence of operators
- Java will automatically convert numerical primitive data types, but results are sometimes surprising
 - take care when mixing integer and floating-point numbers in expressions
- The meaning of an operator is determined by its operands

/

is it integer division or floating-point division?

- *Casting* is the temporary conversion of a variable from its original data type to some other data type.
 - *Like being cast for a part in a play or movie*
- With primitive data types if a cast is necessary from a less inclusive data type to a more inclusive data type it is done automatically.

```
int x = 5;  
double a = 3.5;  
double b = a * x + a / x;  
double c = x / 2;
```

- if a cast is necessary from a more inclusive to a less inclusive data type the cast must be done explicitly by the programmer

- failure to do so results in a compile error.

```
double a = 3.5, b = 2.7;  
int y = (int) a / (int) b;  
y = (int) ( a / b );  
y = (int) a / b; //syntax error
```

Control Structures

- linear flow of control
 - statements executed in consecutive order
- Decision making with if - else statements

```
if (boolean-expression)
    statement;
if (boolean-expression)
{
    statement1;
    statement2;
    statement3;
}
```

A single statement could be replaced by a statement block, braces with 0 or more statements inside

- boolean expressions evaluate to true or false
- Relational Operators: $>$, $>=$, $<$, $<=$, $==$, $!=$
- Logical Operators: $\&\&$, $||$, $!$
 - $\&\&$ and $||$ cause short circuit evaluation
 - if the first part of $p \ \&\& \ q$ is false then q is not evaluated
 - if the first part of $p \ || \ q$ is true then q is not evaluated

//example

```
if ( x <= X_LIMIT && y <= Y_LIMIT )  
    //do something
```

- **if-else:**

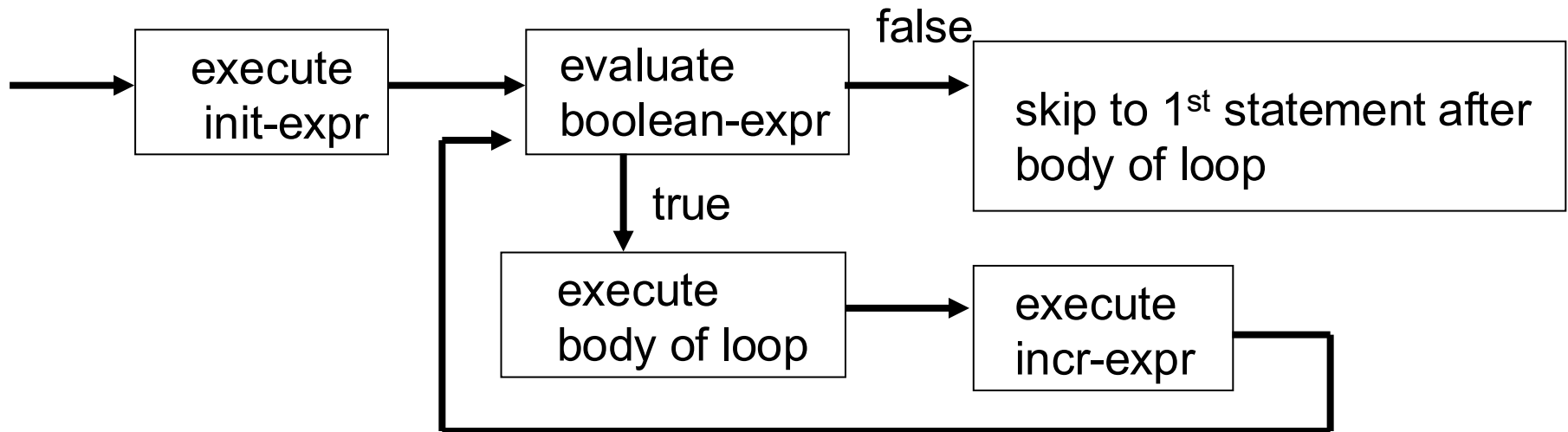
```
if (boolean-expression)  
    statement1;  
else  
    statement2;
```
- **multiway selection:**

```
if (boolean-expression1)  
    statement1;  
else if (boolean-expression2)  
    statement2;  
else  
    statement3;
```
- individual statements could be replaced by a statement block, a set of braces with 0 or more statements
- Java also has the switch statement, but not part of our subset

- for loops

```
for (init-expr; boolean-expr; incr-expr)  
    statement;
```

- init-expr and incr-expr can be more zero or more expressions or statements separated by commas
- statement could be replaced by a statement block



- while loops

```
while (boolean-expression)  
    statement; //or statement block
```

- do-while loop part of language

```
do  
    statement;  
while (boolean-expression) ;
```

- Again, could use a statement block

- break, continue, and labeled breaks

- referred to in the Java tutorial as *branching statements*
- keywords to override normal loop logic
- use them judiciously (which means not much)

Arrays

"Should array indices start at 0 or 1? My compromise of 0.5 was rejected without, I thought, proper consideration." - S. Kelly-Bootle

- Java has built in arrays. a.k.a. native arrays
- arrays hold elements of the same type
 - primitive data types or classes
 - space for array must be dynamically allocated with new operator. (Size is any *integer expression*. Due to dynamic allocation does not have to be constant.)

```
public void arrayExamples()  
{  
    int[] intList = new int[10];  
    for(int i = 0; i < intList.length; i++)  
    {  
        assert 0 >= i && i < intList.length;  
        intList[i] = i * i * i;  
    }  
    intList[3] = intList[4] * intList[3];  
}
```

- all arrays must be dynamically allocated
- arrays have a public, final field called *length*
 - built in size field, no separate variable needed
 - don't confuse length (capacity) with elements in use
- elements start with an index of zero, last index is “length – 1” e.g., array of length 5 runs 0...4
- trying to access a non-existent element results in an `ArrayIndexOutOfBoundsException` (AIOBE)

- Array variables are object variables
- They hold the memory address of an array object
- The array must be dynamically allocated
- All values in the array are initialized (0, 0.0, char 0, false, or null)
- Arrays may be initialized with an initializer list:

```
int[] intList = {2, 3, 5, 7, 11, 13};
```

```
double[] dList = {12.12, 0.12, 45.3};
```

```
String[] sList = {"Olivia", "Kelly", "Isabelle"};
```

- A native array of objects is actually a native array of *object variables*
 - all object variables in Java are really what?
 - Pointers!

```
public void objectArrayExamples()  
{  
    Rectangle[] rectList = new Rectangle[10];  
    // How many Rectangle objects exist?  
  
    rectList[5].setSize(5,10);  
    //uh oh!  
  
    for(int i = 0; i < rectList.length; i++)  
    {  
        rectList[i] = new Rectangle();  
    }  
  
    rectList[3].setSize(100,200);  
}
```

- New in Java 5.0
 - a.k.a. the for-each loop
 - useful short-hand for accessing all elements in an array (or other types of structures) if no need to alter values
 - alternative for iterating through a set of values
- ```
for (Type loop-variable : set-expression)
 statement
```
- logical error (not a syntax error) if try to modify an element in array via enhanced for loop

# Enhanced for loop

```
public static int sumListOld(int[] list)
{
 int total = 0;
 for(int i = 0; i < list.length; i++)
 {
 total += list[i];
 System.out.println(list[i]);
 }
 return total;
}
```

```
public static int sumListEnhanced(int[] list)
{
 int total = 0;
 for(int val : list)
 {
 total += val;
 System.out.println(val);
 }
 return total;
}
```

# Methods

# Methods (1)

- methods are analogous to procedures and functions in other languages
  - local variables, parameters, *instance variables*
  - must be comfortable with variable scope: where is a variable defined?
- methods are the means by which objects are manipulated (objects *state* is changed) - much more on this later
- method header consists of
  - access modifier(**public**, package, protected, **private**)
  - static keyword (optional, class method)
  - return type (void or any data type, primitive or class)
  - method name
  - parameter signature

- local variables can be declared within methods.
  - Their scope is from the point of declaration until the end of the methods, unless declared inside a smaller block like a loop
- methods contain statements
- methods can call other methods
  - in the same class: `foo()` ;
  - methods to perform an operation on an object that is in scope within the method: `obj.foo()` ;
  - static methods in other classes:  
`double x = Math.sqrt(1000) ;`

- the main method is where a stand-alone Java program normally begins execution
- common compile error, trying to call a non static method from a static one

```
public class StaticExample
{
 public static void main(String[] args)
 {
 //starting point of execution
 System.out.println("In main method");
 method1();
 method2(); //compile error;
 }

 public static void method1()
 {
 System.out.println("method 1");
 }

 public void method2()
 {
 System.out.println("method 2");
 }
}
```

- a class may have multiple methods with the same name as long as the parameter signature is unique
  - may not overload on return type
- methods in different classes may have same name and signature
  - this is a type of polymorphism, not method overloading
- if a method has a return value other than void it must have a return statement with a variable or expression of the proper type
- multiple return statements allowed, the first one encountered is executed and method ends
  - style considerations

- a method may have any number of parameters
- each parameter listed separately
- no VAR (Pascal), &, or const & (C++)
- final can be applied, but special meaning
- all parameters are pass by value
- Implications of pass by value???

- A value parameter makes a copy of the argument it is sent.
  - Changes to parameter do not affect the argument.
- A reference parameter is just another name for the argument it is sent.
  - changes to the parameter are really changes to the argument and thus are permanent

# Programming for Correctness

- Java features has a mechanism to check the correctness of your program called *assertions*
- Assertions are statements that are executed as normal statements if assertion checking is on
  - you should always have assertion checking on when writing and running your programs
- Assertions are boolean expressions that are evaluated when reached. If they evaluate to true the program continues, if they evaluate to false then the program halts
- logical statements about the condition or state of your program

- Assertions have the form

```
assert boolean expression : what to output
if assertion is false
```

- Example

```
if ((x < 0) || (y < 0))
{ // we know either x or y is < 0
 assert x < 0 || y < 0 : x + " " + y;
 x += y;
}
else
{ // we know both x and y are not less than zero
 assert x >= 0 && y >= 0 : x + " " + y;
 y += x;
}
```

- Use assertion liberally in your code

- part of style guide

# Assertions Uncover Logical Errors

```
if (a < b)
{ // we a is less than b
 assert a < b : a + " " + b;
 System.out.println(a + " is smaller than " + b);
}
else
{ // we know b is less than a
 assert b < a : a + " " + b;
 System.out.println(b + " is smaller than " + a);
}
```

- Use assertions in code that other programmers are going to use.
- In the real world this is the majority of your code!

# End of week 1!